

EDGE AI BASED SMART EV CHARGING OPTIMIZER

A Multi-Bay Electric Vehicle Charging Station with On-Device Edge AI and Cloud-Based Monitoring

Project Technical Documentation

Internship Project - Emertxe IoT Internship 2026

Submitted by

Rahul Sahu

Bachelor of Technology - Electrical and Electronics Engineering

Project Guide

Lakshmi Dhanyal

Acknowledgement

I would like to express our sincere gratitude to my internship mentor and the Emertxe IoT Internship 2026 program for the guidance, technical resources, and structured curriculum that made this project possible.

We acknowledge the use of open-source platforms and tools that this project depends on, including the Wokwi ESP32 simulator, the PlatformIO build system, the Arduino and ESP32 core libraries, ArduinoJson, PubSubClient, and the ThingsBoard IoT platform, all of which were essential to realizing this system within the scope of a simulated internship environment.

Abstract

Electric vehicle (EV) charging stations with multiple bays face recurring operational problems: uneven bay utilization, congestion during peak hours, lack of demand forecasting, and no centralized visibility into station health. This project presents an Edge AI Based Smart EV Charging Optimizer - a simulated three-bay EV charging station in which each bay is controlled by an ESP32 microcontroller that measures voltage, current, power, energy, and temperature; runs a lightweight Edge AI model locally to predict short-term EV arrival probability and expected session duration; and applies a local optimization algorithm that allows, throttles, or defers charging based on real-time load, predicted demand, and station-wide power constraints.

Each bay publishes telemetry over MQTT to the ThingsBoard IoT cloud platform, where live dashboards, historical data, remote procedure call (RPC) based manual control, and alarms are configured. The Edge AI model consists of a logistic regression classifier for arrival prediction and a linear regression model for session-duration prediction, both trained on a synthetic dataset of 8,000 records and converted into lightweight C/C++ arithmetic so that inference runs directly on the ESP32 without any machine-learning runtime. On a held-out test split, the arrival model achieved 88.9% classification accuracy and the duration model achieved an R-squared of 0.642 with a mean absolute error of approximately 14 minutes.

The system was implemented and validated entirely in a Wokwi circuit simulation integrated with PlatformIO, with all three bays coordinating station-wide power limits through shared ThingsBoard attributes. This document describes the problem motivating the project, the system requirements, the hardware and software architecture, the Edge AI methodology, the optimization algorithm, the implementation, and the results obtained, along with the limitations of the current simulation-based prototype and directions for future work toward a physical deployment.

Table of Contents

Acknowledgement	2
Abstract	3
Table of Contents	4-5
List of Figures	6
List of Abbreviations	7
Chapter 1 - Introduction	8-9
1.1 Background	8
1.2 Existing Approach and Its Limitations	8
1.3 Motivation	8
1.4 Objectives	8
1.5 Scope	8-9
Chapter 2 - Existing Systems and Approaches	10
Chapter 3 - System Requirements	11-12
3.1 Hardware Requirements	11
3.2 Software Requirements	11
3.3 Functional Requirements (Summary)	11-12
3.4 Non-Functional Requirements	12
Chapter 4 - Proposed System	13
4.1 System Overview	13
4.2 System Architecture	13
4.3 Multi-Bay Coordination	13
Chapter 5 - Hardware Design	14-15
5.1 Controller	14
5.2 Sensors and Actuators	14
5.3 Pin Configuration	14
5.4 Circuit Diagram	15
Chapter 6 - Software Design	16-17
6.1 Module Breakdown	16
6.2 Execution Cycles	16
6.3 Communication Protocol	17
Chapter 7 - Edge AI Model	18
7.1 Problem Formulation	18
7.2 Features	18
7.3 Dataset and Training	18
7.4 Measured Performance	18
7.5 On-Device Deployment	18
Chapter 8 - Local Optimization Algorithm	19

8.1 Decision Priorities	19
8.2 Relay Duty-Cycle Control	19
Chapter 9 - Implementation	20-22
9.1 Three-Bay Station Setup	20
9.2 ThingsBoard Dashboard	20-21
9.3 Serial Diagnostics	21-22
Chapter 10 - Testing and Validation	23-24
10.1 Test Cases	23
10.2 Fault and Alarm Testing	23-24
Chapter 11 - Results and Discussion	25
Chapter 12 - Limitations and Future Scope	26
12.1 Limitations	26
12.2 Future Scope	26
Chapter 13 - Conclusion	27
References	28
Appendix A - MQTT Telemetry Schema	29
Appendix B - Source Code and Repository	30

List of Figures

Figure	Caption
Figure 4.1	Overall System Architecture (Edge – MQTT – Cloud)
Figure 5.1	Wokwi Circuit / Wiring Diagram for a Single Bay
Figure 6.1	Software Control Flow (setup and main loop)
Figure 8.1	Local Optimization Decision Flow
Figure 9.1	ThingsBoard Dashboard - Station Overview
Figure 9.2	ThingsBoard Dashboard - Per-Bay Control Panel
Figure 9.3	Serial Monitor Output - Sensor, AI, and Optimization Logs
Figure 10.1	ThingsBoard Alarm Triggered During Fault Testing
Figure 11.1	Model Performance Summary (Arrival Accuracy and Duration Error)

List of Abbreviations

Abbreviation	Meaning
EV	Electric Vehicle
AI	Artificial Intelligence
IoT	Internet of Things
MCU	Microcontroller Unit
MQTT	Message Queuing Telemetry Transport
RPC	Remote Procedure Call
JSON	JavaScript Object Notation
ADC	Analog-to-Digital Converter
GPIO	General Purpose Input / Output
SRS	Software Requirements Specification
R^2	Coefficient of Determination
MAE	Mean Absolute Error

Chapter 1 - Introduction

1.1 Background

The adoption of electric vehicles is increasing the number of charging stations required in commercial and residential settings. A charging station with multiple bays must not only deliver power safely but also manage how that power is shared across bays, since the combined demand of several vehicles charging simultaneously can exceed the station's available capacity. Traditional charging stations apply static, rule-based control - for example, first-come-first-served power allocation - which does not account for how long a session is likely to last, how likely new vehicles are to arrive, or how the station's total load is trending.

1.2 Existing Approach and Its Limitations

Conventional multi-bay charging controllers typically allocate power on a fixed-priority or round-robin basis and expose minimal real-time visibility to the operator. This leads to three recurring issues: bays are not prioritized based on how close a session is to completion, stations are not protected from exceeding grid-safe power limits during peak hours, and faults (sensor failure, overvoltage, a stuck session) are not detected and reported automatically.

1.3 Motivation

This project was undertaken to explore whether a small amount of on-device intelligence - running directly on the same low-cost microcontroller that already measures voltage and current - can meaningfully improve how a multi-bay station allocates power, without requiring a constant connection to a heavyweight cloud inference service. The project was carried out as part of the Emertxe IoT Internship 2026 program, which provided the structured curriculum (sensor interfacing, Wi-Fi/MQTT connectivity, ThingsBoard integration, and Edge AI) used to build the system.

1.4 Objectives

- Simulate a three-bay EV charging station using ESP32 microcontrollers in the Wokwi circuit simulator.
- Measure and calculate per-bay voltage, current, power, energy, and temperature in real time.
- Train and deploy a lightweight Edge AI model that predicts EV arrival probability and expected session duration directly on the ESP32.
- Implement a local optimization algorithm that allows, throttles, or defers charging based on live readings, predictions, and station-wide power limits.
- Publish telemetry to ThingsBoard over MQTT and provide a live dashboard with manual override controls via RPC.
- Validate the system's behavior under normal operation and under simulated fault conditions.

1.5 Scope

Included

- A simulated three-bay station (ESP32 + Wokwi) with sensor acquisition, Edge AI inference, and local optimization.
- MQTT-based telemetry, shared-attribute configuration, and RPC-based remote control through ThingsBoard.
- A synthetic dataset and trained logistic/linear regression models for arrival and duration prediction.

Not Included

- Certification of a physical, grid-connected commercial charger.

- High-power hardware design (the project uses low-voltage simulated sensing, not a real EV battery or AC/DC charging circuit).
- Deployment on live utility infrastructure.

Chapter 2 - Existing Systems and Approaches

Multi-bay EV charging control strategies generally fall into one of the categories summarized below. This comparison motivates the approach adopted in this project, which combines a lightweight predictive model with a deterministic, explainable optimization rule set rather than relying purely on either static rules or a opaque model.

Approach	Advantages	Limitations
Fixed-schedule charging	Simple to implement; predictable	Ignores real-time demand and grid conditions
First-come-first-served allocation	Fair in arrival order	Does not prioritize sessions nearing completion
Cloud-based AI control	Can use large, complex models	Requires constant connectivity; higher latency
Edge AI with local optimization (this project)	Works with intermittent connectivity; fast local decisions; explainable rule layer on top of the prediction	Limited to lightweight models that fit on a microcontroller

Chapter 3 - System Requirements

3.1 Hardware Requirements

Component	Specification	Purpose
Microcontroller	ESP32 DevKit (Wokwi simulated)	Main per-bay controller: sensing, Edge AI, MQTT
Voltage sensing	Potentiometer, mapped 0–250 V	Simulated bay voltage input
Current sensing	Potentiometer, mapped 0–32 A	Simulated bay current input
Temperature/Humidity sensor	DHT22	Bay environmental monitoring and fault detection
Relay module	1-channel relay	Switches simulated charging current on/off
Status LEDs	Red / Yellow / Green	Local visual bay status (fault / charging / free)
Push buttons	2 per bay	Simulated EV plug-in and plug-out events

3.2 Software Requirements

Software / Library	Purpose
PlatformIO + Arduino framework	Firmware build and upload toolchain
Wokwi simulator	Circuit-level ESP32 simulation
ArduinoJson	JSON construction/parsing for MQTT payloads
PubSubClient	MQTT client for ESP32
DHT sensor library / Adafruit Unified Sensor	DHT22 driver
ThingsBoard Cloud	Telemetry storage, dashboards, alarms, RPC
Python (pandas, scikit-learn)	Synthetic dataset generation and model training

3.3 Functional Requirements (Summary)

1. The system shall sample bay sensors (voltage, current, power, energy, temperature) on a configurable interval, by default every 5 seconds.
2. The system shall detect EV plug-in and plug-out events and update bay status accordingly.
3. The system shall run an Edge AI inference cycle, by default every 15 minutes, producing a predicted arrival probability and predicted remaining session duration.
4. The system shall run a local optimization cycle, by default every 30 seconds, producing an ALLOW, THROTTLE, or DEFER decision per bay.
5. The system shall publish telemetry to ThingsBoard over MQTT on a configurable interval.
6. The system shall accept shared-attribute configuration (station load limit, overload current, peak tariff window, prediction threshold, and related safety thresholds) from ThingsBoard.

7. The system shall support remote RPC commands to set relay state, set a manual throttle level, clear a manual override, and report immediate status.
8. The system shall detect and report fault conditions: sensor failure, overvoltage, overcurrent, and a session stuck beyond its maximum allowed duration.

3.4 Non-Functional Requirements

- Reliability - the firmware must automatically recover Wi-Fi and MQTT connectivity after an outage without manual intervention.
- Responsiveness - Edge AI inference must complete in well under one second on the ESP32.
- Safety - any detected fault condition must immediately override normal optimization and de-energize the relay.
- Scalability - the same firmware image must support any bay in the station, with only its ThingsBoard device token and bay identifier differing between units.

Chapter 4 - Proposed System

4.1 System Overview

The proposed system is organized around three pillars: Input (sensing bay voltage, current, temperature, and plug status), Intelligence (an on-device Edge AI model that predicts near-term EV arrival probability and expected session duration from recent and historical patterns), and Action (a local optimization algorithm that converts those predictions, together with live readings and station-wide context, into an ALLOW, THROTTLE, or DEFER decision for the bay's relay).

4.2 System Architecture

Each bay's ESP32 forms the edge layer: it samples sensors, runs Edge AI inference, executes the optimization algorithm, and drives the relay and status LEDs directly. The communication layer is MQTT, used to publish telemetry and subscribe to shared-attribute updates and RPC requests. The cloud layer is ThingsBoard, which stores historical telemetry, renders live dashboards, evaluates alarm rules, and relays operator commands back to the device as RPCs.

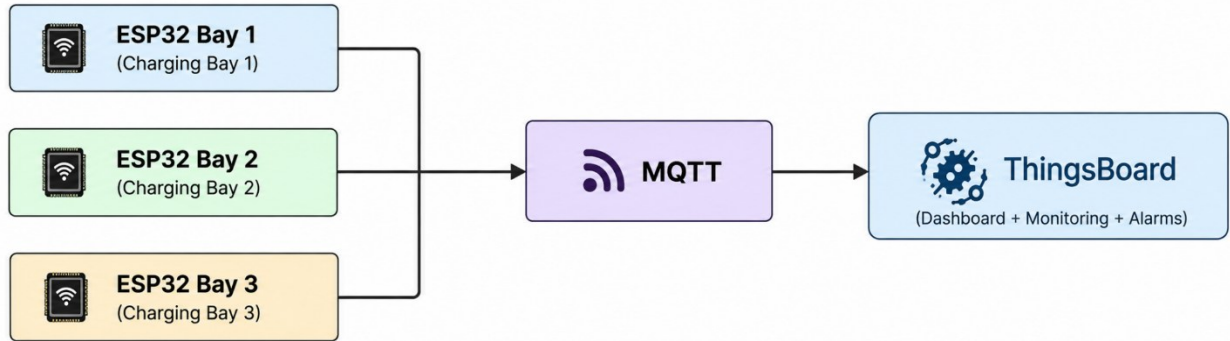


Figure 4.1 - Overall system architecture: Edge (ESP32) – MQTT – ThingsBoard Cloud

4.3 Multi-Bay Coordination

The three bays (BAY1, BAY2, BAY3) operate independently but share station-wide context through ThingsBoard shared attributes: each bay's locally measured power is combined with the power reported by its neighbors to check whether the station-wide load limit is being approached, and each bay's predicted session duration is compared against the median of its neighbors' predicted durations so that sessions closer to completion are prioritized for full power during a station-wide overload.

Chapter 5 - Hardware Design

5.1 Controller

Each bay is controlled by an ESP32 DevKit module (simulated in Wokwi as the esp32-devkit-c-v4 board), running at 240 MHz with built-in Wi-Fi, which is used for both MQTT connectivity and the device's internal clock synchronization (NTP) for time-of-day features used by the Edge AI model.

5.2 Sensors and Actuators

Voltage and current are simulated using two potentiometers read through the ESP32's analog-to-digital converter and linearly mapped to realistic ranges (0–250 V and 0–32 A respectively). A DHT22 sensor provides temperature (and humidity, unused by the current logic) and also serves as the system's fault-detection sensor: a failed read latches a sensor-fault flag that forces the bay into a safe DEFER state. A relay module switches the simulated charging current, and three LEDs (green, yellow, red) give an immediate local indication of FREE, CHARGING, and FAULT states respectively. Two push buttons simulate an EV being plugged in and unplugged.

5.3 Pin Configuration

ESP32 Pin	Connected Device	Function
GPIO 34	Voltage potentiometer	Analog input - simulated voltage
GPIO 35	Current potentiometer	Analog input - simulated current
GPIO 15	DHT22 data line	Temperature/humidity input
GPIO 26	Relay module (IN)	Charging relay control output
GPIO 32	Plug-in push button	EV plug-in event input
GPIO 33	Plug-out push button	EV plug-out event input
GPIO 18	Green LED	Bay FREE indicator
GPIO 19	Yellow LED	Bay CHARGING indicator
GPIO 21	Red LED	Bay FAULT indicator

5.4 Circuit Diagram

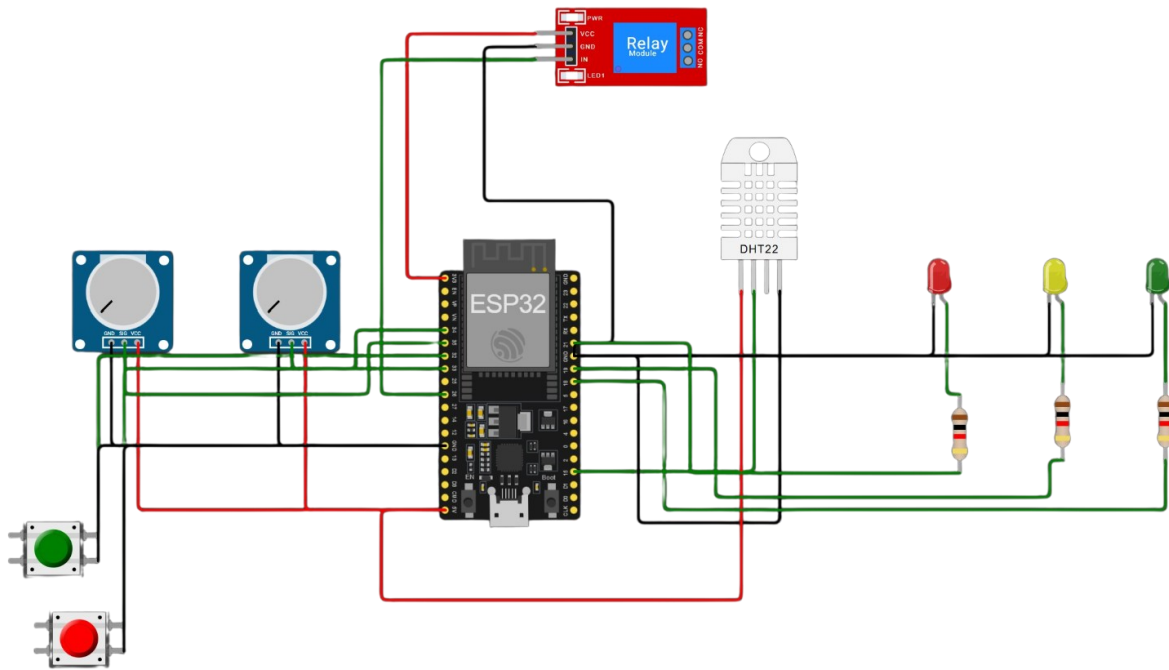


Figure 5.1 - Wokwi circuit/wiring diagram for a single charging bay

Chapter 6 - Software Design

6.1 Module Breakdown

Module	Responsibility
State	Holds all live bay variables, configuration defaults, and energy accumulation
Peripherals	Sensor sampling, plug-in/plug-out button handling, voltage/current mapping
Network	Wi-Fi and MQTT connection management, incoming message dispatch
Telemetry	Builds and publishes the per-bay telemetry JSON payload
Attributes	Requests and applies shared configuration from ThingsBoard; publishes client attributes
Edge AI	Feature construction and on-device inference for arrival and duration prediction
Optimization	The decision algorithm (ALLOW/THROTTLE/DEFER) and relay duty-cycle control
RPC	Handles remote commands: relay control, manual throttle, status request, override clear
Main	Orchestrates all modules on independent timing cycles

6.2 Execution Cycles

Rather than a single fixed loop, the firmware runs four independent timing cycles so that each subsystem runs at the cadence appropriate to it: sensor sampling and telemetry publishing every 5 seconds, local optimization every 30 seconds, and Edge AI inference every 15 minutes (reusing the most recent prediction between cycles). The relay duty-cycle and status LEDs are re-evaluated on every loop iteration so that partial throttle levels are applied as a genuine time-proportional duty cycle rather than a single snapshot decision.

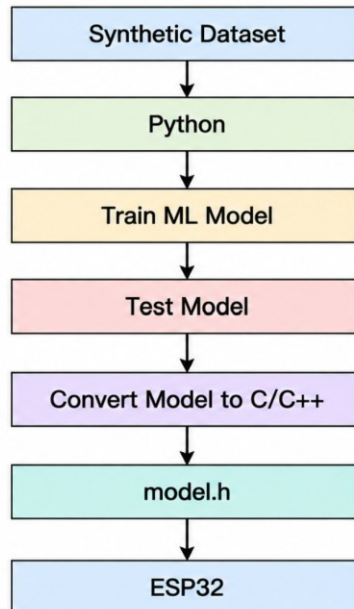


Figure 6.1 - Software control flow across `setup()` and the main loop

6.3 Communication Protocol

Each device authenticates to ThingsBoard using a per-device MQTT access token and publishes to the standard ThingsBoard telemetry topic. Shared attributes are requested on connect and updated whenever ThingsBoard pushes a change. RPC requests arrive on a per-request topic and are dispatched by method name to the appropriate handler in the RPC module, with a JSON response published back to the matching response topic.

Chapter 7 - Edge AI Model

7.1 Problem Formulation

Two related but separate predictions are made for each bay on every inference cycle: the probability that a new EV will arrive at that bay within the near-term window, and, if the bay is currently charging, the estimated remaining duration of that session. Both predictions feed directly into the optimization algorithm described in Chapter 8.

7.2 Features

Both models share the same seven input features, listed in their defined order:

#	Feature	Description
1	hourOfDay	Current hour (0–23), from NTP-synchronized time
2	dayOfWeek	Current day of week (0–6)
3	bayOccupied	1 if the bay is currently charging, else 0
4	recentAvgCurrent	Most recent measured current for the bay
5	sessionElapsedMin	Minutes elapsed in the current session, if any
6	neighborBaysOccupied	Count of occupied neighboring bays in the station
7	historicalArrivalRate	A time-of-day/day-of-week base arrival rate pattern

7.3 Dataset and Training

A synthetic dataset of 8,000 records was generated to reflect realistic arrival and duration patterns (higher arrival rates during 08:00–10:00 and 18:00–21:00, lower overnight, with a weekend adjustment). The arrival model is a logistic regression classifier trained on whether an EV arrived within the prediction window; the duration model is a linear regression trained only on currently-occupied records, predicting actual session duration in minutes. Both were trained with an 80/20 train-test split using scikit-learn.

7.4 Measured Performance

Model	Metric	Value
Arrival (Logistic Regression)	Test accuracy	0.8888
Duration (Linear Regression)	Test R^2	0.6421
Duration (Linear Regression)	Test Mean Absolute Error	13.99 minutes

7.5 On-Device Deployment

Both trained models were converted into plain C/C++ floating-point arithmetic (equivalent to the output of m2cgen) so that inference requires no external machine-learning runtime on the ESP32. Measured inference time for a complete cycle (both models) is on the order of microseconds to low milliseconds, well within the requirement that inference not interfere with the device's real-time sensing and networking duties.

Chapter 8 - Local Optimization Algorithm

8.1 Decision Priorities

The optimization algorithm evaluates conditions in strict priority order so that safety conditions always take precedence over efficiency or convenience decisions:

9. Sensor fault detected - force DEFER and de-energize the relay.
10. Overvoltage detected - force DEFER and de-energize the relay.
11. Local overcurrent while charging - THROTTLE to 50 percent.
12. Session running longer than the configured maximum duration - THROTTLE as a stuck-session safeguard.
13. Bay is FREE, it is within the peak-tariff window, and predicted arrival probability is below the configured threshold - DEFER to avoid energizing a bay unlikely to be used imminently during an expensive period.
14. Bay is CHARGING and station-wide total power exceeds the configured limit - compare this bay's predicted remaining duration against the median of its neighbors'; THROTTLE to 70 percent if this session is not closer to completion than its neighbors, otherwise ALLOW at full power.
15. Otherwise - ALLOW at full power.

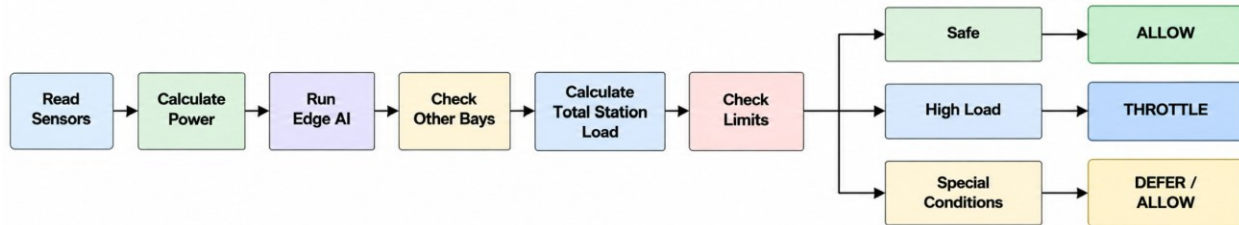


Figure 8.1 - Local optimization decision flow for a single bay

8.2 Relay Duty-Cycle Control

A throttle level between 0 and 100 percent is translated into physical relay behavior: 100 percent keeps the relay continuously energized, 0 percent keeps it continuously de-energized, and any intermediate value is applied as a time-proportional duty cycle over a fixed 2-second window, so that, for example, a 50 percent throttle level energizes the relay for 1 second out of every 2.

Chapter 9 - Implementation

9.1 Three-Bay Station Setup

Three identical firmware images were built, differing only in each device's ThingsBoard access token and bay identifier (BAY1, BAY2, BAY3), which are isolated into a per-device, uncommitted configuration file so that the same source code is shared across all three bays.

9.2 ThingsBoard Dashboard

A live operations dashboard was configured in ThingsBoard showing, per bay: voltage, current, power, temperature, energy consumed, bay status, predicted arrival probability, predicted remaining duration, load decision, optimization reason, and throttle level, alongside time-series charts comparing all three bays. Manual controls (a relay on/off switch and a throttle slider) are bound to the device's RPC methods, and a station configuration panel allows the shared attributes (station load limit, overload current, peak tariff window, and related thresholds) to be edited directly from the dashboard.

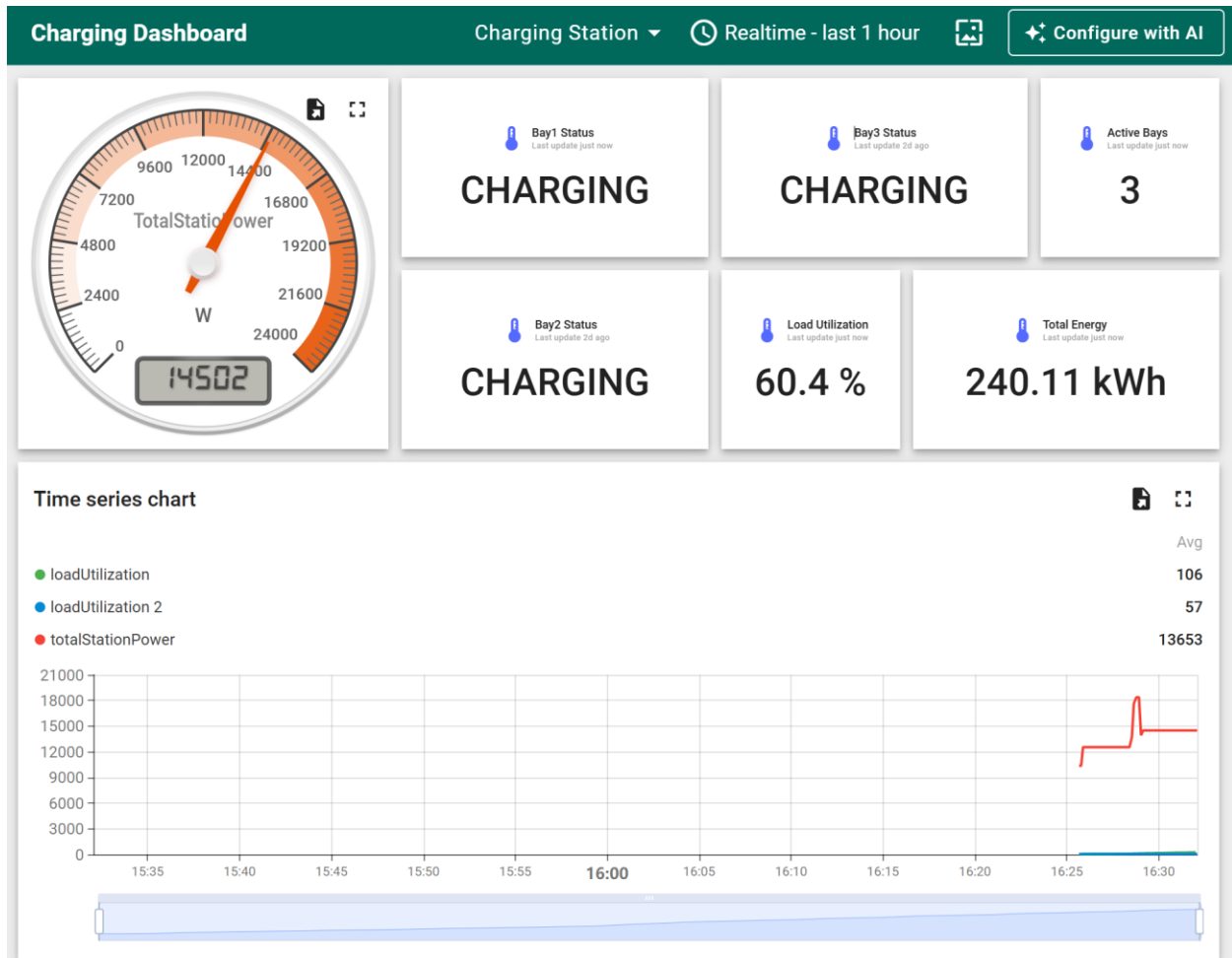


Figure 9.1 - ThingsBoard dashboard, station overview

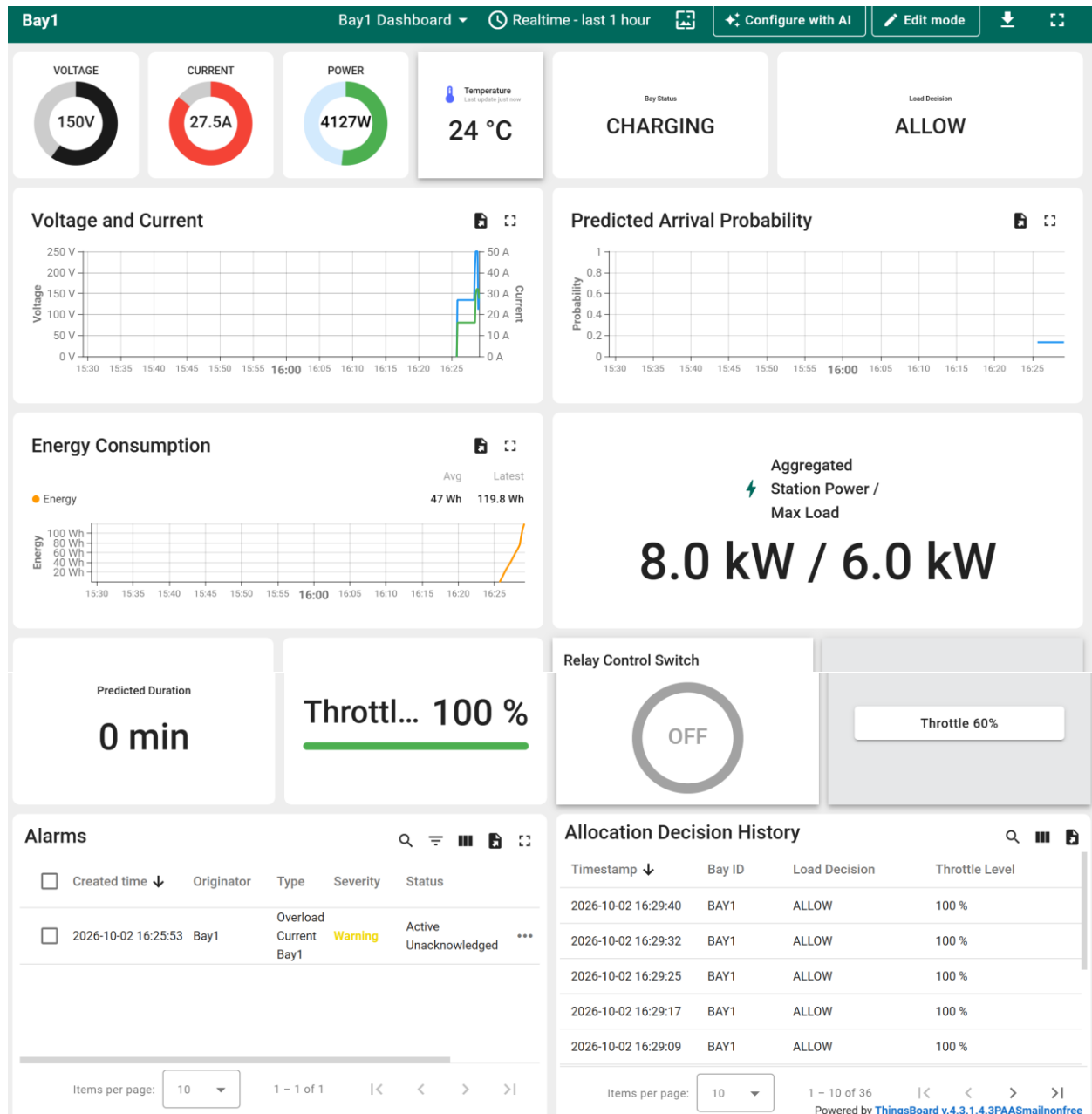


Figure 9.2 - ThingsBoard dashboard, per-bay control panel

9.3 Serial Diagnostics

Each stage of the per-cycle pipeline (sensor sampling, Edge AI inference, optimization decision, and telemetry publish) logs a concise status line to the serial monitor during development, which was used throughout implementation and testing to confirm correct behavior before relying on the cloud dashboard alone.

```

=====
BAY1 EV CHARGING OPTIMIZER
=====
[WiFi] Connecting...
[WiFi] Connected. IP = 10.13.37.2
[MQTT] Connecting to ThingsBoard... connected.
[ATTR >>] {"bayId":"BAY1","firmwareVersion":"1.0.0","ipAddress":"10.13.37.2"}
[ATTR] Shared attributes requested.
[AI] Arrival probability = 0.135
[AI] Predicted remaining duration = 0 min
[AI] Inference time = 0.036 ms
=====
BAY1 READY
=====
[MQTT <<] v1/devices/me/attributes/response/1 -> {"shared":{"predictionThreshold":3000,"maxStationLoadW":3000,"overloadCurrentA":3000}}
[ATTR] Shared attributes updated.
  maxStationLoadW = 3000.00
  overloadCurrentA = 3000.00
  peakTariff = 18-21
  predictionThreshold = 3000.00
  neighborBaysOccupied = 0
[MQTT >>] {"bayId":"BAY1","voltage":0,"current":0,"power":0,"temperature":24,"energyWh":0,"bayStatus":"FREE","predictedArrivalProb":0.135,"predictedDurationMin":0,"loadDecision":"ALLOW","optimizationReason":"NORMAL","throttleLevel":0,"overloadActive":false,"overvoltageActive":false,"sensorFault":false,"stuckSession":false,"manualOverrideActive":false,"stationTotalPowerW":0,"neighborBaysOccupied":0,"stationBayCount":1,"sessionElapsedMin":0}
[BUTTON] EV plugged in -> CHARGING -> RELAY ON
[MQTT >>] {"bayId":"BAY1","voltage":0,"current":0,"power":0,"temperature":24,"energyWh":0,"bayStatus":"CHARGING","predictedArrivalProb":0.135,"predictedDurationMin":0,"loadDecision":"ALLOW","optimizationReason":"NORMAL","throttleLevel":100,"overloadActive":false,"overvoltageActive":false,"sensorFault":false,"stuckSession":false,"manualOverrideActive":false,"stationTotalPowerW":0,"neighborBaysOccupied":0,"stationBayCount":1,"sessionElapsedMin":0}

```

Figure 9.3 - Serial monitor output showing sensor, AI, and optimization logs

Chapter 10 - Testing and Validation

10.1 Test Cases

Test ID	Condition	Expected Behavior	Result
T01	Normal charging, no faults	Bay allows full power (ALLOW, 100%)	PASS
T02	Current exceeds overload threshold while charging	Bay throttles to 50%, overload flag set	PASS
T03	Voltage exceeds overvoltage threshold	Bay enters FAULT, relay de-energized	PASS
T04	DHT22 read failure	Sensor fault flag set, bay DEFERs	PASS
T05	Session exceeds max session duration	Stuck-session flag set, bay throttles	PASS
T06	Station total power exceeds station limit	Lower-priority bay throttles to 70%	PASS
T07	Wi-Fi disconnected mid-operation	Device reconnects automatically without manual reset	PASS
T08	Manual RPC setRelayState(true/false)	Relay follows command; manual override flag set	PASS
T09	Manual RPC setThrottle(level)	Relay duty cycle follows the commanded level in real time	PASS
T10	RPC getStatus with no parameters	Immediate JSON status response returned	PASS

10.2 Fault and Alarm Testing

Fault conditions (overcurrent, overvoltage, and sensor failure) were each triggered individually by adjusting the simulated sensor inputs in Wokwi, and the corresponding ThingsBoard alarm rules were confirmed to fire and clear correctly as each condition was introduced and then resolved.

Alarms



☐	Created time ↓	Originator	Type	Severity	Status	
☐	2026-10-02 16:25:53	Bay1	Overload Current Bay1	Warning	Active Unacknowledged	...

Items per page: 10 ▼

1 – 1 of 1



Figure 10.1 - ThingsBoard alarm triggered during fault testing

Chapter 11 - Results and Discussion

The system met its primary objectives within the scope of a circuit-level simulation. The Edge AI model's 88.9% arrival-prediction accuracy and duration-prediction error of approximately 14 minutes are reasonable given that both targets were generated from a synthetic dataset with deliberately added noise, and they are sufficient to meaningfully bias the optimization algorithm's DEFER and THROTTLE decisions toward genuinely low-demand or near-complete sessions rather than acting on raw sensor readings alone.

The station-wide coordination behavior (comparing a bay's predicted remaining duration against its neighbors' median before throttling) produced the intended effect of preserving full power for sessions closer to completion during a simulated station overload, rather than throttling bays uniformly or arbitrarily.

Edge AI Model Performance Summary

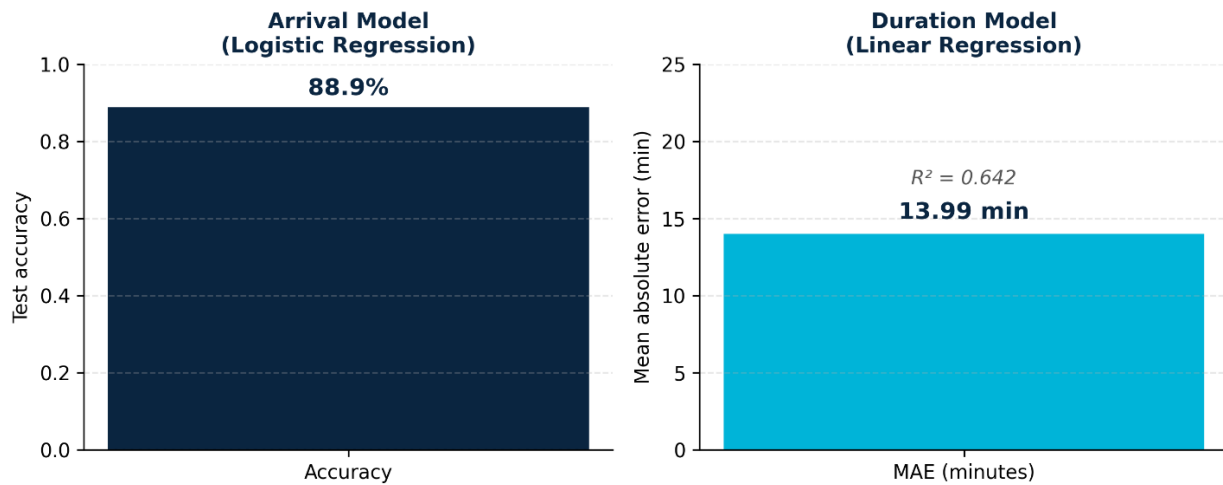


Figure 11.1 - Model performance summary: arrival accuracy and duration error

Chapter 12 - Limitations and Future Scope

12.1 Limitations

- The system is validated in circuit simulation (Wokwi); it has not been deployed on physical EV-charging hardware or connected to a real grid.
- Voltage and current are simulated via potentiometers rather than measured through calibrated current/voltage sensors (e.g., ACS712, voltage divider with real EV-side hardware).
- Both Edge AI models are trained on a synthetic dataset rather than on data collected from real charging sessions.
- The MQTT connection to ThingsBoard currently uses plaintext (non-TLS) transport, which is acceptable for the Wokwi prototype but not for production deployment.

12.2 Future Scope

- Deploy the firmware on physical ESP32 hardware connected to real current/voltage sensors and a real relay-controlled load.
- Replace the synthetic training dataset with data collected from the deployed station over time, and periodically retrain both models.
- Enable TLS-secured MQTT and move device credentials to a hardware-backed secure element.
- Extend the optimization algorithm to actively recommend a specific bay to an arriving EV (load balancing at arrival time) rather than only reacting after a bay is occupied.
- Add a small local display (e.g., OLED) for on-site status, as scoped but not implemented in the current design.

Chapter 13 - Conclusion

This project set out to determine whether lightweight, on-device Edge AI could improve multi-bay EV charging coordination without depending on constant cloud connectivity. A three-bay station was simulated using ESP32 microcontrollers, each running real-time sensor acquisition, a trained logistic/linear regression Edge AI model, and a priority-ordered local optimization algorithm, all integrated with the ThingsBoard IoT platform for monitoring, configuration, and remote control. The resulting system successfully demonstrated safe fault handling, station-wide load coordination, and prediction-informed charging decisions, meeting the objectives set out in Chapter 1 within the scope of a simulated prototype. The limitations identified in Chapter 12 outline a clear and realistic path from this prototype toward a physically deployed system.

References

- [1] Espressif Systems, "ESP32 Series Datasheet," Technical Documentation, 2024.
- [2] ThingsBoard, "ThingsBoard Documentation - MQTT API, Widgets, and Rule Engine," <https://thingsboard.io/docs/>, accessed 2026.
- [3] PlatformIO, "PlatformIO Documentation," <https://docs.platformio.org/>, accessed 2026.
- [4] Aosong Electronics, "DHT22 Digital-Output Humidity and Temperature Sensor Datasheet."
- [5] Pedregosa, F. et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.
- [6] Wokwi, "Wokwi Simulator Documentation," <https://docs.wokwi.com/>, accessed 2026.

Appendix A - MQTT Telemetry Schema

Each bay publishes the following keys to the ThingsBoard telemetry topic on every telemetry cycle:

Key	Type	Description
bayId	string	Device identity (BAY1 / BAY2 / BAY3)
voltage	number	Measured voltage (V)
current	number	Measured current (A)
power	number	Instantaneous power (W)
temperature	number	Bay temperature (°C)
energyWh	number	Cumulative energy this session (Wh)
bayStatus	string	FREE / CHARGING / FAULT
predictedArrivalProb	number	Edge AI arrival probability (0–1)
predictedDurationMin	number	Edge AI predicted remaining duration (min)
loadDecision	string	ALLOW / THROTTLE / DEFER / MANUAL_*
optimizationReason	string	Reason code for the current decision
throttleLevel	number	Applied throttle level (0–100)
stationTotalPowerW	number	Combined power across all bays (W)

Appendix B - Source Code and Repository

The complete firmware (per bay), the Edge AI training scripts, the generated dataset, and the ThingsBoard dashboard configuration are maintained in the project's GitHub repository. Refer to the repository's README for build and flashing instructions using PlatformIO.